

log(x) -- returns the natural logarithm of x
sin(x) -- returns the sine of x
sqrt(x) -- returns the square root of x

Here is an example:

```
{
  x = 5.3241
  y = int(x)
  printf "truncated value is ", y
}
```

Output: truncated value is 5

String functions in AWK

1. *length(string)*: Calculates the length of a string.
2. *index(string, substring)*: Returns the first position of the substring within a string. For example, in *x=index("programming", "gra")*, x returns the value 4.
3. *substr()*: Extracts the substring from a string. The two different ways to use it are: *substr(string, position)* and *substr(string, position, length)*.

Here is an example:

```
{
  x = substr("methodology",3)
  y = substr("methodology",5,4)
  print "substring starts at " x
  print " substring of length " y
}
```

The output of the above code is:

Substring starts at hodology
 Substring of length dolo

4. *sub(regex, replacement string, input string)* or *gsub(regex, replacement string, input string)*
sub(/Ben/, " Ann ", \$0): Replaces Ben with Ann (first occurrence only).
gsub(/is/, " was ", \$0): Replaces all occurrences of 'is' with 'was'.
5. *match(string, regex)*

```
{
  x=match($0,/^[0-9]/)      #find all lines that
start with digit
  if(x>0)                  #x returns a value > 0 if there's a
match
    print $0
}
```

6. *toupper()* and *tolower()*: This is used for convenient conversions of case, as follows:

```
{
  print toupper($0)        #converts entire file to uppercase
}
```

User defined functions in AWK

AWK allows us to define our own functions, which enables reusability of the code. A large program can be divided into functions, and each one can be written/tested independently.

The syntax is:

```
Function Function_name (parameter list)
{
    Function body
}
```

The following example finds the largest of two numbers. The program's name is *awfuncnt*.

```
{
  print large($1,$2)
}
function large(m,n)
{
    return m>n ? m : n
}
```

Input file is: doc

100 400

Running the script: *awk -f awfuncnt doc*

400

Associative arrays

AWK allows one-dimensional arrays, and their size and elements need not be declared. An array index can be a number or a string.

The syntax is:

```
arrayName[index] = value
```

Index entry is associated with an array element, so AWK arrays are known as associative arrays.

For example, *dept[\$2]* indicates an element in the second column of the file and is stored in the array, *dept*.

```
Program name : awkarray
BEGIN{print "Eg of arrays in awk"}
{
  dept[$2]
  for (x in dept)
  {
    print a[x]
  }
}
```